Piano di azione – Alveare

Dal prototipo a un sistema reale, costruito con Claude Code

Versione 1.0 · ottobre 2026 · “Alveare” è un nome di lavoro, da confermare con la community.

0. Come usare questo piano

Il piano è diviso in sette fasi (0–6). Ogni fase indica:

- **obiettivo**: cosa deve esistere alla fine;
- **attività**: il lavoro da fare, in ordine;
- **criteri di accettazione**: come si verifica che la fase è chiusa;
- **prompt per Claude Code**: testo pronto da incollare in una sessione;
- **rischi**: cosa può andare storto e come accorgersene presto.

Punto di partenza. La cartella `04_prototipo` contiene già il nucleo della rete locale offline, funzionante e testato (28 test superati, demo completa con processi reali): bundle firmati, registro locale, nodi con agent card, orchestratore con ricette parallele e failover, politica di rete solo LAN, validazione locale con rapporto firmato. Il piano parte da lì.

Ordine delle fasi. Prima la rete locale (fasi 1–3), che ha valore anche da sola. Poi il primo modello specialistico (fase 4, in parallelo alla 3). Solo dopo la piattaforma condivisa (fase 5) e la prova sul campo (fase 6). Questo ordine segue l’analisi delle potenzialità: il valore sta nello sbloccare l’uso dell’IA dove la privacy lo impedisce, e i due vincoli decisivi sono la semplicità d’uso e il numero di volontari che preparano dati.

1. Obiettivo del prototipo reale

In circa **16 settimane** con 1–2 persone e Claude Code:

1. una rete locale **realmente scollegata da internet** in un ufficio pilota, con 3 compiti specialistici su modelli veri a licenza OSI (traduzione it→en, riassunto, correzione);
2. un launcher installabile da chi non è tecnico, anche senza connessione;
3. un primo **torneo di specializzazione** sulla piattaforma condivisa, con validazione distribuita e una release firmata scaricabile nella rete locale.

Fuori perimetro (per ora): training distribuito sincrono di un unico modello, token o criptovalute, app mobile nativa, modelli di immagini in produzione (solo sperimentazione in fase 4b).

2. Preparare Claude Code

1. Installa Claude Code seguendo la documentazione ufficiale (https://docs.claude.com/en/docs/claude-code/overview) e aprilo nella cartella del prototipo.
2. Il file `CLAUDE.md` è già pronto: contiene mappa del codice, comandi e invarianti (offline per costruzione, nessun modello senza verifica, solo licenze OSI, segreti mai in chiaro). Claude Code lo carica a ogni sessione: aggiornalo alla fine di ogni fase.

3. Comandi personalizzati già presenti in `.claude/commands/`: - `/fase N` – pianifica e implementa la fase N di questo piano; - `/verifica` – test, demo, `alveare doctor`, controllo di chiamate di rete e segreti; - `/nuova-ricetta <descrizione>` – nuovo flusso di lavoro per l'orchestratore; - `/nuovo-compito <nome>` – nuovo modello specialistico nella rete di esempio.
4. Subagente in `.claude/agents/revisore-offline.md`: revisione di sicurezza (solo lettura) per ogni modifica a rete, firme, autenticazione e caricamento dei modelli.

Ciclo di lavoro consigliato per ogni sessione

1. Entra in plan mode e lancia `/fase N`: Claude legge il codice e propone un piano senza modificare nulla.
2. Rispondi alle domande aperte e approva il piano (o correggilo).
3. Implementazione a piccoli passi: test prima, poi codice, `pytest` a ogni passo.
4. `/verifica` e, se la modifica tocca rete o firme, chiedi la revisione al subagente `revisore-offline`.
5. Commit su un ramo dedicato alla fase; aggiorna `CLAUDE.md` e `docs/SPECIFICHE.md`.

Regole pratiche

- Una fase per ramo git, richieste di merge piccole e leggibili.
- Mai disattivare un test per "far passare" una modifica: se un invariante va cambiato, si cambia prima `CLAUDE.md`.
- Le decisioni importanti (scelta dei modelli, formati, licenze) vanno scritte in `docs/decisioni/` con data e motivazione.

---

3. Laboratorio minimo

Ruolo	Hardware consigliato	Note
Orchestratore + nodo principale	mini-PC o PC con 16–32 GB di RAM (meglio GPU da 12–16 GB)	ospita anche il registro locale
Nodo leggero	Raspberry Pi 5 da 8 GB	compiti brevi: correzione, classificazione
Client e secondo nodo	un laptop qualsiasi	prova del parallelismo su due nodi
Training (fasi 4–5)	GPU da 12–24 GB, anche a noleggio	QLoRA di modelli fino a 8B in circa 6 GB di VRAM
Rete	switch senza collegamento a internet	per i test di isolamento

---

4. Le fasi

Fase 0 – Fondamenta (settimana 1)

Obiettivo. Repository pubblico pronto ad accogliere contributori.

Attività

1. Repository git con il contenuto di `04_prototipo`; testo integrale della licenza Apache-2.0 in `LICENSE`.
2. Integrazione continua che esegue `pytest` a ogni modifica.
3. `CONTRIBUTING.md` (come proporre modifiche, Developer Certificate of Origin), `CODE_OF_CONDUCT.md`, `SECURITY.md` (come segnalare vulnerabilità).
4. Cartella `docs/decisioni/` con la prima decisione: perimetro del prototipo e licenze ammesse.

5. Verifica del nome del progetto (marchi, domini, omonimie).

Accettazione. Integrazione continua verde; un contributore esterno riesce a eseguire test e demo seguendo solo il README.

Prompt per Claude Code

> Leggi CLAUDE.md. Prepara il repository per contributori esterni: aggiungi una pipeline di integrazione continua che installa il pacchetto con le dipendenze di sviluppo ed esegue pytest su Python 3.10, 3.11 e 3.12; scrivi CONTRIBUTING.md, CODE_OF_CONDUCT.md e SECURITY.md in italiano; crea docs/decisioni/0001-perimetro.md con il perimetro descritto nella sezione 1 del piano. Non modificare il codice applicativo.

Rischi. Nessuno rilevante; è la fase in cui fissare le regole prima che arrivino contributi.

Fase 1 – Rete locale con modelli veri (settimane 2–4)

Obiettivo. La rete funziona in una LAN isolata con modelli veri e si installa senza internet.

Attività

1. **Modelli veri.** Collegare i nodi a llama-swap/llama-server (configurazione d'esempio in `examples/llama-swap.yaml`). Scegliere 3 modelli iniziali a licenza OSI, verificando la licenza al momento della scelta (le licenze cambiano tra versioni): per la traduzione un modello dedicato piccolo con licenza Apache-2.0 (ad esempio della famiglia Hy-MT2); per riassunto e correzione un modello piccolo generalista Apache-2.0 da 1–4B.
2. **Streaming** delle risposte (SSE) da backend a nodo a orchestratore a client.
3. **Altri tipi di compito**: endpoint per embedding e trascrizione audio (Whisper via llama-swap), così il campo `kind` del manifest diventa operativo.
4. **Scoperta automatica dei nodi** in LAN (mDNS/DNS-SD) in alternativa all'elenco statico, con conferma manuale del nuovo nodo da parte dell'amministratore.
5. **Osservabilità locale**: log strutturati, registro delle operazioni (chi ha chiesto cosa, a quale nodo, con quale bundle) che non lascia mai la rete, metriche di latenza per compito.
6. **Installazione offline**: pacchetto con tutte le dipendenze Python (wheel scaricate su una macchina collegata e installate con `--no-index`), script di installazione per Linux, macOS e Raspberry Pi OS.
7. **Prova di isolamento**: test automatico che fallisce se un processo della rete apre connessioni verso indirizzi non LAN (ad esempio controllando le connessioni attive durante la demo).

Accettazione

- Con il cavo internet scollegato, la ricetta `traduci-e-riassumi` su un contratto di 10 pagine termina con modelli veri e distribuisce le parti su almeno due nodi.
- `alveare doctor` pulito su tutte le macchine; la prova di isolamento passa.
- Installazione completa su una macchina nuova senza internet in meno di 30 minuti.

Prompt per Claude Code

> /fase 1. Inizia dal punto 1: aggiungi un test di integrazione opzionale (saltato se manca llama-server) che avvia llama-swap con un modello GGUF piccolo indicato da una variabile d'ambiente ed esegue la ricetta traduci-e-riassumi. Poi implementa lo streaming SSE mantenendo compatibile la risposta non in streaming. Ogni nuova chiamata HTTP deve passare da client_factory e da check_urls.

Rischi. Prestazioni dei modelli piccoli in italiano inferiori alle attese → misurarle subito con i casi di validazione e annotare i risultati in `docs/decisioni/`.

Fase 2 – Bundle, registro e aggiornamenti offline (settimane 4–5)

Obiettivo. La catena di fiducia è completa: dalla firma del progetto al caricamento sul nodo, con aggiornamenti portati a mano e possibilità di revoca.

Attività

1. **Compatibilità con OpenSSF Model Signing (OMS)**: firmare i bundle anche nel formato OMS usando coppie di chiavi (la variante senza chiavi via Sigstore richiede la rete e non è adatta alla verifica offline).
2. **Pacchetto di aggiornamento su supporto rimovibile**: bundle + chiavi pubbliche + elenco di revoca firmato + indice con hash; comando `alveare import /percorso/usb` che verifica tutto prima di copiare.
3. **Revoca**: elenco firmato di bundle e chiavi revocati; un nodo disabilita i compiti revocati all'avvio.
4. **Rotazione delle chiavi** del progetto con firma incrociata della chiave nuova da parte della vecchia.
5. **Agent card conforme A2A**: allineare il formato alla specifica A2A pubblicata, mantenendo i campi Alveare come estensione.
6. **Elenco delle dipendenze (SBOM)** generato a ogni release del software.

Accettazione

- Test di manomissione su ogni elemento del pacchetto (pesi, manifest, firma, indice, elenco di revoca).
- Un bundle revocato viene disabilitato su tutti i nodi dopo l'import del nuovo pacchetto.
- L'agent card supera il validatore dello schema A2A.

Prompt per Claude Code

> /fase 2. Parti dalla revoca: progetta il formato dell'elenco di revoca firmato in docs/SPECIFICHE.md, scrivi i test (bundle revocato → compito disabilitato con motivo leggibile nell'agent card), poi implementa. Fai rivedere il risultato al subagente revisore-offline.

Rischi. Complessità della gestione chiavi per utenti non tecnici → il launcher (fase 6) deve nasconderla del tutto.

Fase 3 – Orchestratore intelligente (settimane 6–8)

Obiettivo. L'utente scrive una richiesta in linguaggio naturale; l'orchestratore sceglie o costruisce la ricetta giusta, la esegue in parallelo e sa quando fermarsi a chiedere.

Attività

1. **Router**: un modello piccolo trasforma la richiesta in una chiamata strutturata (quale ricetta, con quali input), con output vincolato a uno schema JSON. Modelli tipo Needle sono un buon riferimento per questo ruolo, ma lo standard resta aperto.
2. **Pianificatore**: per le richieste che nessuna ricetta copre, un modello più grande sul nodo principale genera una ricetta (grafo di passi) nello stesso formato YAML; la ricetta passa dalla validazione dello schema e da un elenco di compiti consentiti prima di essere eseguita, come nell'approccio di LLMCompiler (pianificatore, smistamento, esecuzione parallela).
3. **Confidenza e conferma**: tre fasce (esegui, chiedi conferma, rifiuta); soglie calibrate sui casi di validazione locali (regressione isotonica) invece che sulla confidenza grezza del modello.

4. **Escalation interna**: se il nodo piccolo non è sicuro, il passo viene ripetuto sul nodo più grande della stessa rete, mai su servizi esterni.
5. **Interfaccia**: endpoint compatibile OpenAI sull'orchestratore, così strumenti esistenti (editor, chat locali) possono usare la rete senza modifiche.

Accettazione

- Su 50 richieste di prova scritte da persone diverse, almeno il 90% è instradato alla ricetta corretta.
- Nessuna ricetta generata viene eseguita senza validazione; i casi a bassa confidenza chiedono conferma.

Prompt per Claude Code

> /fase 3. Prima prepara il banco di prova: un file JSONL con 50 richieste in italiano e la ricetta attesa, e un comando `alveare valuta-router` che misura l'accuratezza. Solo dopo implementa il router con output vincolato a uno schema JSON. Il pianificatore deve produrre ricette nel formato di docs/SPECIFICHE.md e passare da Recipe.model_validate prima dell'esecuzione.

Rischi. I modelli piccoli calibrano male la propria confidenza, specie fuori dall'inglese → soglie da misurare sui dati locali, mai fissate a priori.

Fase 4 – Primo modello specialistico (settimane 6–10, in parallelo alla fase 3)

Obiettivo. Un modello piccolo, specializzato e compresso che batte il proprio modello base su un compito reale e gira su hardware modesto.

Attività

1. **Scelta del compito**: riassunto o correzione di testi professionali italiani (rientrano nel circa 27% di richieste di lavoro che consistono nel trasformare un testo dato).
2. **Dati**: 2.000 esempi di addestramento e 500 di test. Con il ritmo di OpenAssistant (circa 46 compiti per volontario) servono circa 54 volontari. Integrare con dati sintetici generati da un modello insegnante **aperto ed eseguito in locale** (niente modelli chiusi, per non creare problemi di licenza).
3. **Strumenti per i dati**: piccola interfaccia web locale per scrivere e revisionare esempi, con deduplicazione e controllo delle sovrapposizioni tra training e test.
4. **Training**: QLoRA con Unsloth su GPU da 8–24 GB; più tentativi con iperparametri diversi.
5. **Compressione**: conversione GGUF Q4_K_M per i nodi generici; prova di quantizzazioni più spinte solo se la valutazione lo consente; misura su Raspberry Pi 5 e su un mini-PC.
6. **Bundle firmato** con il report di valutazione nel manifest.
7. (*4b, sperimentale*) un adattatore LoRA per immagini su base Apache-2.0 per verificare la pipeline diffusion (training da 12–24 GB di VRAM).

Accettazione

- Sul test privato, il modello specializzato supera il modello base della stessa taglia (soglia da fissare in `docs/decisioni/` prima del training, per evitare di spostarla dopo).
- Gira sul nodo leggero entro una latenza accettabile per il compito (da definire con l'ufficio pilota).

Prompt per Claude Code

> /fase 4. Crea una cartella training/ separata dal pacchetto alveare: script per preparare i dati (JSONL), controllare duplicati e sovrapposizioni train/test, lanciare un fine-tuning QLoRA configurabile da YAML, convertire in GGUF e creare il bundle firmato con i risultati di validazione nel campo eval del manifest. Niente download automatici: i modelli base si indicano con percorsi locali.

Rischi. Dati scarsi o ripetitivi → misurare la varietà del dataset; se i volontari non arrivano, è il primo segnale che il vincolo principale del progetto è la community, non la tecnologia.

Fase 5 – Pilota della piattaforma condivisa (settimane 10–14)

Obiettivo. Un torneo completo: più persone addestrano candidati in parallelo, i validatori li valutano con test privati, la release vincente viene firmata e diventa scaricabile nelle reti locali.

Attività

1. **Scheda del torneo**: compito, modello base, dati pubblici di addestramento, formato del candidato (adattatore o bundle), scadenze (4–7 giorni per round, come i tornei di Gradients).
2. **Invio dei candidati**: ogni partecipante pubblica un bundle candidato firmato con la propria chiave; la ricetta di training dei vincitori diventa pubblica, così il round successivo parte dalla migliore.
3. **Validazione distribuita**: i validatori scaricano i candidati, eseguono `alveare validate` sui propri casi privati (anche da reti offline, esportando solo il rapporto firmato) e caricano i rapporti.
4. **Aggregazione**: mediana dei punteggi validi, peso per reputazione del validatore, soglia minima di validatori per round, riesecuzione a campione di alcuni casi pubblici per individuare chi bara.
5. **Release**: il candidato vincente viene firmato con la chiave del progetto, pubblicato nel registro pubblico e nel pacchetto per supporti rimovibili.
6. **Crediti non commerciabili** (sul modello dei kudos di AI Horde): priorità nei round successivi e accesso anticipato alle release, senza mercato secondario.

Accettazione

- Almeno 10 candidati e 20 validatori indipendenti in un round; release firmata pubblicata e importata in una rete offline; i risultati del torneo sono riproducibili da terzi.

Prompt per Claude Code

> /fase 5. Progetta in docs/SPECIFICHE.md il formato della scheda del torneo e del candidato. Implementa `alveare torneo` con i sottocomandi apri, invia, valida, aggrega, rilascia, riusando validate.py e bundle.py. L'aggregazione deve scartare rapporti non firmati, applicare la soglia minima di validatori e restituire mediana e dispersione. Scrivi test con validatori onesti e disonesti simulati.

Rischi. Partecipazione bassa senza incentivi economici (AI Horde, puramente volontario, conta poche decine di worker attivi alla volta); manipolazione dei punteggi → soglie minime e riesecuzioni a campione.

Fase 6 – Launcher e prova sul campo (settimane 12–16)

Obiettivo. Una persona non tecnica installa e usa la rete in un ufficio vero.

Attività

1. **Launcher**: interfaccia web servita dall'orchestratore (nessuna dipendenza esterna) con: elenco dei compiti, invio di documenti, scelta della ricetta, stato dei nodi, import del pacchetto da supporto rimovibile in un clic.

2. **Configurazione guidata** di nuovi nodi: rilevamento hardware, proposta dei compiti adatti alla macchina, generazione delle chiavi API senza mostrarle.
3. **Pilota** in 1–3 organizzazioni (studio professionale, PMI, ente) per 3–4 settimane.
4. **Misure**: tempo di installazione, quota di richieste di testo servite localmente, tempi di risposta, errori, soddisfazione degli utenti.

Accettazione

- Installazione completata dall'ufficio pilota senza assistenza diretta in meno di 30 minuti.
- Almeno il 25% delle richieste di testo dell'ufficio servite dalla rete locale.

Prompt per Claude Code

> /fase 6. Crea un'interfaccia web minimale servita dall'orchestratore (HTML e JavaScript senza CDN, tutto incluso nel pacchetto): pagina per caricare un documento, scegliere una ricetta e vedere i risultati con la traccia; pagina di stato dei nodi; pagina di import del pacchetto di aggiornamento. Nessuna risorsa caricata da internet. Aggiungi test per gli endpoint usati dall'interfaccia.

Rischi. La mancanza di competenze è la prima barriera citata dalle imprese (oltre il 70% di quelle che hanno valutato l'IA in UE) → la semplicità del launcher è un requisito, non un dettaglio.

5. Punti di decisione

Momento	Domanda	Se la risposta è no
Fine fase 1	La rete funziona offline con modelli veri e latenze accettabili?	ridurre i compiti o puntare a hardware un gradino sopra
Fine fase 4	Il modello specializzato batte il base sul test privato?	rivedere dati e compito prima di aprire la piattaforma
Fine fase 5	Ci sono almeno 10 trainer e 50 volontari per i dati?	ripensare gli incentivi o restringere i progetti
Fine fase 6	L'ufficio pilota usa davvero la rete (≥ 25% delle richieste di testo)?	migliorare il launcher prima di allargare

6. Rischi principali

Rischio	Segnale	Mitigazione
Pochi volontari per i dati	dataset fermo sotto i 1.000 esempi dopo un mese	dati sintetici da insegnante aperto, partnership con università e associazioni
Licenze dei modelli che cambiano	nuova versione con licenza non OSI	fissare le versioni, archiviare i pesi, preferire basi completamente aperte
Prestazioni insufficienti in italiano	punteggi bassi sui casi di validazione	specializzazione mirata, modelli multilingue completamente aperti
Manipolazione dei tornei	validatori con punteggi anomali	mediana, reputazione, riesecuzioni a campione, soglia minima
Software che "chiama casa"	connessioni in uscita nella prova di isolamento	test automatico di isolamento, firewall in uscita, dipendenze verificate
Sostenibilità del progetto	contributori in calo	fondazione o ente neutrale, servizi di installazione per aziende

7. Governance, licenze, aspetti legali

- **Codice**: Apache-2.0, contributi con Developer Certificate of Origin.
- **Modelli e dati**: solo licenze OSI per i pesi; dataset con licenza compatibile e provenienza documentata; niente output di modelli chiusi nei dati di addestramento.
- **Ente neutrale**: a regime, una fondazione o un progetto ospitato da un ente esistente (il modello della Linux Foundation per MCP e A2A è un riferimento).
- **Privacy**: la rete locale non trasferisce dati; documentarlo è un vantaggio per la conformità al GDPR delle organizzazioni che la adottano.
- **Verifica legale** prima del primo rilascio pubblico di modelli: obblighi applicabili ai fornitori di modelli nell'UE e responsabilità sulle release firmate.

8. Calendario riassuntivo

Settimane	Fase	Risultato
1	0 – Fondamenta	repository pubblico con integrazione continua
2–4	1 – Rete con modelli veri	demo offline con 3 compiti reali, installazione senza internet
4–5	2 – Catena di fiducia	aggiornamenti da USB, revoca, firma OMS
6–8	3 – Orchestratore intelligente	richieste in linguaggio naturale, conferme
6–10	4 – Primo modello specialistico	bundle firmato che batte il modello base
10–14	5 – Pilota della piattaforma	primo torneo con release pubblica
12–16	6 – Launcher e pilota	uso reale in 1–3 organizzazioni